# Dal testo grezzo al testo pulito

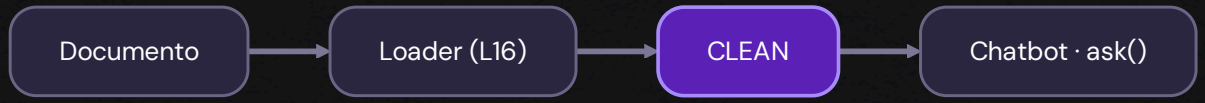
Text Cleaning & Pre-processing — dare al chatbot un testo che capisce davvero

Modulo M2 · Lezione 17 · 2 luglio 2026



# Dove siamo: il testo è grezzo

A L16 abbiamo imparato a **caricare** i file e a darli al chatbot con `ask()`. Ma ricordate l'HTML: il loader "funzionava", eppure la risposta del modello **peggiorava** — il *noise* (menu, footer) finiva nel prompt. Il testo che esce dai loader è **grezzo**.




Oggi aggiungiamo il pezzo che mancava: **ripulire** il testo prima di darlo al chatbot, così la risposta è migliore.

> Aggiungiamo al toolkit `aikit/clean.py` — e lo proviamo col chatbot su **tre documenti veri** e sporchi.

# 1. Il "testo sporco"



# Il banco di prova: chiediamo al chatbot

Riprendiamo `ask()` di L16: mette il testo del documento nel prompt e fa una domanda al modello. È il nostro metro di giudizio — **la risposta è buona?**

**Testo sporco → risposta peggiore**

Il modello si perde tra menu, note e header ripetuti; a volte cita proprio quelli. Su un documento lungo, il rumore può persino **non farci stare** il contenuto nel prompt.

**Testo pulito → risposta migliore**

Nel prompt entra **solo il contenuto**: la risposta è centrata, e spendi meno token.

```
python ask.py sample_docs/wikipedia_nlp.html "che cos'è l'NLP?"
```

→ confronta la risposta sul testo **sporco** vs **pulito**.



# Su un documento non si vede

Su un volantino di una pagina il rumore costa due centesimi e il modello se la cava. Il punto è un altro: **quasi mai** un'app lavora su un documento solo.

**1 documento corto**

Il boilerplate è poca roba: ci sta nel prompt, costa poco. Si perdona.

**Tanti documenti, o lunghi**

Ogni documento porta il **suo** boilerplate: il rumore si moltiplica. Il **costo** cresce a ogni domanda (L14) e il **context window** (L10) si riempie di spazzatura invece che di contenuto.

Pulire **una volta a monte** ripaga su tutta la pipeline — non a ogni domanda.



# Mettiti nei panni del modello

Il modello legge **solo** il testo che gli passiamo, e non sa cosa è menu e cosa è contenuto: per lui è tutto uguale. Come decidiamo cosa togliere? Con una domanda semplice.

**«Se dovessi rispondere io alla domanda, questo pezzo di testo mi servirebbe?»**

Se la risposta è no — un menu, un footer, *"Published as a conference paper"* ripetuto a ogni pagina — è **rumore**, e va tolto.

**Non lo vorrei →** menu, header/footer ripetuti, note, link

**Lo vorrei →** titoli, paragrafi, elenchi: aiutano a orientarsi

Non serve una lista di regole a memoria: serve quel **punto di vista**. Le tecniche di oggi sono solo i modi per eseguirlo.



# Le famiglie di noise

Il testo grezzo è pieno di elementi di **impaginazione** e di **contenitore** che non sono contenuto. Quattro famiglie ricorrenti (con esempi dai nostri tre documenti veri).

BOILERPLATE

Licenze, header/footer, menu, "Produced by", sezioni di coda (*Note*, *Collegamenti esterni*).

IMPAGINAZIONE

Righe spezzate a ~70 caratteri, sillabazione `lit-⏎erature`, header ricorrenti, numeri di pagina.

WHITESPACE / UNICODE

Spazi e tab multipli, righe vuote a raffica, accenti compositi, caratteri di controllo invisibili.

RESIDUI HTML

Citazioni `[n]`, tag-noise sopravvissuti al loader di L16.



# Normalizzare, con criterio

Normalizzare vuol dire portare il testo in **forma canonica**. Ma non tutto va appiattito: alcune scelte hanno un trade-off.

### Sempre

Unicode **NFC** (accenti uniformati), collasso di whitespace, rimozione dei caratteri di controllo invisibili. Sono a costo zero e non perdono nulla.

### Dipende — es. lowercase

Uniforma, ma **perde segnale** (sigle, nomi propri). Con i modelli di oggi spesso **non serve**: si lascia il case.

Normalizzare non vuol dire *appiattire*. Ogni step va scelto sapendo cosa toglie.



# Cosa non rimuovere

C'è un rischio opposto al noise: **pulire troppo**. Un cleaning aggressivo butta via il contenuto insieme al rumore.

### 🚩 Il rischio opposto

Non buttare la **struttura utile** — titoli, paragrafi, elenchi: aiutano il modello a capire. E non toccare i **metadata** di L16 ( `source` , `page` : da dove arriva il documento).

> Il cleaning tocca `.text` , **non** i metadata. L'obiettivo: togliere il rumore, **non** il segnale.



# 2. `clean.py`: la pipeline componibile



# L’idea: una lista di funzioni

Una pipeline di cleaning **non è un framework**. È una lista di funzioni pure `str → str`, applicate in ordine. Ogni step fa **una** cosa ed è testabile da solo — stesso pattern di composizione di L15.

```python
def clean(text: str, steps: list) -> str:
    for step in steps:          # una pipeline è solo una lista di funzioni
        text = step(text)
    return text
```

Il potere è nella **composizione**: componi step diversi per documenti diversi. Nessuna "funzione magica pulisci-tutto".

`scripts/aikit/clean.py`



# Il codice di oggi

Un modulo `clean.py` dentro il toolkit `aikit/`, che cresce lezione dopo lezione. Riusa i loader e il chatbot delle lezioni precedenti.

```
scripts/
├── aikit/
│   ├── clean.py          # gli step + il runner clean() + le PIPELINES per tipo di file
│   └── loaders/          # i loader di L16 (riuso) + txt.py
├── ask.py                # l'unico runner: before/after (offline) e chatbot (L15/L16)
├── sample_docs/          # i 3 documenti reali "sporchi ma fixabili"
└── esercizio/            # remove_urls (stub + soluzione + check.py)
```

`python ask.py` mostra il before/after **senza chiave**. `python ask.py <file> "domanda"` chiede al chatbot (serve la chiave). Un documento nuovo? Si tocca solo `clean.py`.



# Gli step base

Le fondamenta: whitespace, unicode, caratteri invisibili. Ognuna è una funzione minuscola con una sola responsabilità.

```python
def normalize_whitespace(text: str) -> str:
    text = text.replace("\t", " ")
    text = re.sub(r"[ ]{2,}", " ", text)                    # spazi multipli → uno
    text = "\n".join(l.rstrip() for l in text.split("\n"))  # niente trailing
    return re.sub(r"\n{3,}", "\n\n", text).strip()          # max una riga vuota

def normalize_unicode(text: str) -> str:
    return unicodedata.normalize("NFC", text)               # accenti uniformati
```

scripts/aikit/clean.py



# `reflow_paragraphs` → Frankenstein

Il singolo `\n` dentro un paragrafo è un a-capo di **impaginazione**, non una vera interruzione → diventa spazio. La riga vuota (paragrafo) resta.

prima

```
You will rejoice to hear that no
disaster has accompanied the
commencement of an enterprise
which you have regarded…
```

dopo

```
You will rejoice to hear that no disaster has accompanied
the commencement of an enterprise which you have
regarded…
```

Più `keep_between_markers("*** START", "*** END")` per la licenza Gutenberg: **7742 → ~1600 righe**, il testo torna a scorrere.



# `dehyphenate` → PDF arXiv

I PDF impaginati spezzano le parole a fine riga. Le riuniamo — ma **solo tra lettere**, per non toccare `e-mail` o `2019-2020`.

```python
def dehyphenate(text: str) → str:
    # 'lit-\nerature' → 'literature'
    return re.sub(r"([A-Za-zÀ-ÿ])-\n([A-Za-zÀ-ÿ])", r"\1\2", text)
```

prima: Sci-⏎ence · cor-⏎pus · au-⏎thorship

dopo: Science · corpus · authorship

scripts/aikit/clean.py · sample_docs/arxiv_1905.00075.pdf



# Header ricorrenti e numeri di pagina

Su ogni pagina si ripete *"Published as a conference paper at ICLR 2019"*, e compaiono numeri di pagina. Una riga **corta e ripetuta** è impaginazione, non contenuto.

```
def remove_repeated_lines(min_repeats: int = 3, max_len: int = 80):
    def step(text: str) → str:
        conteggio = Counter(l.strip() for l in text.split("\n") if l.strip())
        ripetute = {r for r, n in conteggio.items()
                    if n ⩾ min_repeats and len(r) ⩽ max_len}
        return "\n".join(l for l in text.split("\n") if l.strip() not in ripetute)
    return step
```

Più `remove_page_numbers` (via le righe fatte solo di cifre): **6 header su 7 rimossi**.



# Il caso-limite: il cleaning è euristico

Sei header su sette. Ne resta **uno**: su pagina 1 il numero è incollato all'header, quindi la riga è **unica** e sfugge al conteggio delle ripetizioni.

```
"1 Published as a conference paper at ICLR 2019"
```

Non è un bug: è la **natura del mestiere**. Nessuna pipeline è perfetta — si guarda il residuo e si decide **se vale la pena** inseguirlo. Spesso no.

💬 Domanda per voi: come lo togliereste? E *conviene* scrivere codice più fragile per un solo caso?



# HTML: citazioni e sezioni di coda → Wikipedia

Dopo il loader HTML di L16 restano le note `[n]` e le sezioni finali (*Note*, *Bibliografia*, *Collegamenti esterni*): boilerplate che non serve al contenuto.

```python
def remove_citation_markers():
    return lambda text: re.sub(r"\[\d+\]", "", text)

def drop_sections(headings: list):
    def step(text: str) -> str:
        lines = text.split("\n")
        for k, l in enumerate(lines):
            if l.strip() in headings:      # tronca alla prima sezione di coda
                return "\n".join(lines[:k]).strip()
        return text
    return step
```

> Stesso runner `clean()`, step diversi: ogni tipo di documento ha il suo noise. Qui **−12.6%** di caratteri (in meno nel prompt del chatbot).



# Una pipeline per tipo di documento

Il punto della lezione: non una funzione unica, ma **mattoncini** che componi in base al noise che hai davanti. Stesso `clean()`, tre ricette — la mappa `PIPELINES` in `clean.py`.

| Documento | Step della pipeline ( `PIPELINES[".ext"]` ) |
|---|---|
| `.txt` (Gutenberg) | `keep_between_markers` · `normalize_unicode` · `reflow_paragraphs` |
| `.pdf` (arXiv) | `dehyphenate` · `remove_repeated_lines` · `remove_page_numbers` · `normalize_whitespace` |
| `.html` (Wikipedia) | `remove_citation_markers` · `drop_sections` · `normalize_whitespace` |

Documento nuovo = **una funzione + una riga** in `PIPELINES`, tutto in `clean.py`. `ask.py` la usa in automatico.



# 3. Il tuo turno + dove andiamo



# Esercizio: `remove_urls`

Prima di dare il testo al chatbot, lo ripuliamo. Molti documenti reali hanno **URL** (link nei footer, nelle note) che non servono a rispondere e sprecano spazio nel prompt. Scrivi lo step che li toglie e agganciarlo alla pipeline.

```python
def remove_urls(text: str) -> str:
    # TODO: togli gli URL (http://..., https://..., www...) lasciando il resto
    # suggerimento: una regex → r"(https?://\S+|www\.\S+)"
    raise NotImplementedError("Completa remove_urls")
```

Verifica con `python check.py` → `[OK]/[FAIL]` per requisito, **senza chiave API**. La soluzione è in `soluzione.py`.

`scripts/esercizio/mia_pulizia.py`



# Ora il testo è pulito

Testo grezzo → testo pulito → **risposta migliore del chatbot**, e meno token spesi. È il pezzo che mancava dopo il caricamento di L16.




> Sabato (**L18**): stessa pipeline **load → clean** su un dataset reale sporco — incluso il caso in cui il parsing nativo non basta e serve un parser gestito (**LlamaParse**).

# Grazie

Una pipeline di cleaning è solo **una lista di funzioni** — `clean(text, steps)`

Testo grezzo → testo pulito → chatbot più preciso. Sabato lo mettiamo alla prova su dati reali sporchi (**L18**).

